

TUGAS STRUKTUR DATA

Sorting Lanjutan dan Binary Tree

Nama : Lionel Jevon Chrismana Putra

NIM : 25091397019

Kelas : 2025A

BAGIAN I: ANALISIS & DESAIN ALGORITMA SORTING LANJUTAN

Soal a. Ruang & Distribusi Sort

1. Mengapa Radix Sort Standar Tidak Memenuhi Batasan Memori $O(1)$ pada Modul A

Implementasi standar Radix Sort menggunakan array of 10 queues (bucket) untuk mendistribusikan elemen berdasarkan digit pada setiap pass. Setiap bucket berupa antrian (queue) yang dapat menampung hingga n elemen dalam kasus terburuk. Akibatnya, total alokasi memori tambahan yang dibutuhkan adalah $O(10 \times n) = O(n)$. Hal ini secara langsung melanggar batasan $O(1)$ extra space pada Modul A yang hanya mengizinkan alokasi ruang tambahan konstan di luar array input.

Lebih rinci, setiap pass Radix Sort membutuhkan:

- a) Array count/frekuensi berukuran 10 (untuk digit 0-9): $O(1)$, diterima.
- b) Array output sementara berukuran n untuk menampung hasil distribusi sebelum disalin kembali: $O(n)$, ditolak.

Kombinasi keduanya menjadikan total ruang tambahan Radix Sort adalah $O(n)$, bukan $O(1)$.

2. Mekanisme tmpArray Tunggal pada Improved Merge Sort

Pada Improved Merge Sort (Listing 12.2 & 12.4), hanya dialokasikan satu array sementara tmpArray berukuran n di awal eksekusi, sebelum rekursi dimulai. Array ini kemudian dioper sebagai parameter ke seluruh pemanggilan rekursif berikutnya sehingga tidak pernah ada alokasi ulang di dalam rekursi.

Mekanisme ini berbeda secara fundamental dengan versi berbasis slice. Pada versi slice, setiap pemanggilan rekursif membuat subarray fisik baru (misalnya `arr[left:mid]` dan `arr[mid+1:right]`), sehingga total memori yang dialokasikan sepanjang pohon rekursi mencapai $O(n \log n)$ karena pada setiap level rekursi terdapat salinan data berukuran $O(n)$.

Dengan satu tmpArray tunggal:

- a) Tidak ada subarray baru yang dibuat di setiap rekursi.
- b) Operasi merge hanya menulis ke tmpArray lalu menyalin kembali ke arr.
- c) Total overhead ruang tambahan tetap $O(n)$ dari satu tmpArray, ditambah $O(\log n)$ untuk stack rekursi.

Dibandingkan versi slice yang membutuhkan $O(n \log n)$, versi tmpArray tunggal jauh lebih hemat memori, meskipun tetap $O(n)$ dan secara teknis tidak memenuhi batasan $O(1)$ murni.

3. Strategi Modifikasi Radix Sort agar $O(1)$ Extra Space

Jika ingin memanfaatkan sifat distribusi digit Radix Sort tanpa melanggar batasan $O(1)$, strategi paling realistis adalah American Flag Sort (in-place Radix Sort). Algoritma ini bekerja dalam dua pass per digit:

Pass 1: Hitung frekuensi setiap digit (hanya butuh array count[10], $O(1)$).

Pass 2: Lakukan pemindahan in-place menggunakan teknik cycle-following sehingga elemen dipindahkan langsung tanpa buffer output tambahan.

Dampak terhadap kompleksitas waktu: Secara teoritis tetap $O(d \times n)$, namun dalam praktik konstanta lebih besar karena akses memori tidak berurutan (non-sequential) sehingga cache miss lebih sering. Stabilitas juga tidak terjamin tanpa modifikasi tambahan. Dengan demikian, American Flag Sort adalah trade-off: $O(1)$ extra space terpenuhi, tetapi performa aktual lebih lambat dari Radix Sort standar untuk data besar.

Soal b. Linked List & Pointer Manipulation

1. Mekanisme `_splitLinkedList()` dengan Fast-Slow Pointer

Fungsi `_splitLinkedList()` menemukan titik tengah linked list dalam satu traversal tunggal menggunakan teknik fast-slow pointer (juga dikenal sebagai Floyd's tortoise and hare). Mekanismenya adalah sebagai berikut:

- Inisialisasi: `midPoint = head` (bergerak 1 langkah per iterasi) dan `curNode = head.next` (bergerak 2 langkah per iterasi).
- Loop: `while curNode and curNode.next: midPoint = midPoint.next; curNode = curNode.next.next`
- Ketika `curNode` mencapai akhir list (None atau node terakhir), `midPoint` berada tepat di posisi tengah.

Alasan matematis: Karena `curNode` bergerak dua kali lebih cepat dari `midPoint`, ketika `curNode` sudah menempuh seluruh list (n langkah), `midPoint` baru menempuh $n/2$ langkah, yaitu tepat di tengah. Setelah loop selesai, `right_head = midPoint.next` dan `midPoint.next = None` untuk memutus list menjadi dua bagian.

2. Dummy Node & Tail Reference pada `_mergeLinkedLists()`

Teknik dummy node dan tail reference memungkinkan operasi merge linked list tanpa alokasi memori tambahan karena:

- Dummy node hanyalah satu node statis (bukan alokasi dinamis berulang) yang berfungsi sebagai sentinel/header palsu. Node ini tidak mengambil data baru, hanya menyimpan pointer.

- b) Tail reference selalu menunjuk ke node terakhir dari hasil merge. Operasi penambahan node ke hasil cukup dengan `tail.next = node_terpilih`; `tail = tail.next`, yang merupakan operasi pointer $O(1)$.
- c) Tidak ada node baru yang dibuat. Hanya pointer `.next` yang dimodifikasi untuk menghubungkan ulang node-node yang sudah ada.

Kompleksitas ruang turun menjadi $O(\log n)$ karena satu-satunya penggunaan memori tambahan adalah call stack rekursi. Kedalaman rekursi Merge Sort pada linked list adalah $O(\log n)$ karena setiap level membagi list menjadi dua bagian yang lebih kecil. Tidak ada array tambahan, tidak ada node baru, sehingga ruang ekstra hanya berasal dari stack rekursi itu sendiri.

Soal c. Quick Sort Worst-Case & Pivot Strategy

1. Perilaku Pointer pada Data Descending

Ketika Quick Sort diterapkan pada data yang sudah terurut terbalik (descending) dan pivot selalu dipilih sebagai elemen pertama (`arr[first]`), maka setiap kali partisi dijalankan pivot yang terpilih adalah elemen terbesar dalam subarray saat itu.

Perilaku penanda left dan right pada `partitionSeq()`:

- a) Left bergerak dari kiri ke kanan mencari elemen $\geq$ pivot. Karena semua elemen lebih kecil dari pivot (data descending), left tidak menemukan elemen $\geq$ pivot dan terus bergerak hingga melewati right.
- b) Right bergerak dari kanan ke kiri mencari elemen $\leq$ pivot. Karena pivot adalah elemen terbesar, right langsung berhenti di posisi pivot sendiri.
- c) Akibatnya, partisi selalu menghasilkan subarray kiri berukuran $n-1$ dan subarray kanan berukuran 0.

Dengan pembagian $n-1$ dan 0 setiap kali, kedalaman rekursi menjadi $O(n)$ karena terdapat n level rekursi. Total perbandingan menjadi $n + (n-1) + (n-2) + \dots + 1 = O(n^2)$. Inilah kasus terburuk Quick Sort.

2. Strategi Pivot yang Lebih Robust: Median-of-Three

Strategi yang diusulkan adalah Median-of-Three: memilih median dari tiga kandidat, yaitu `arr[first]`, `arr[mid]`, dan `arr[last]`, sebagai pivot. Caranya:

- a) Bandingkan ketiga nilai tersebut.
- b) Tempatkan yang terkecil di `arr[first]`, terbesar di `arr[last]`, dan median di `arr[mid]` atau langsung gunakan sebagai pivot.
- c) Tukar pivot ke posisi `arr[first]` sebelum menjalankan logika partisi standar.

Dengan Median-of-Three, pada data descending pivot yang terpilih adalah nilai tengah dari tiga elemen sehingga partisi menghasilkan pembagian yang lebih seimbang dan menghindari $O(n^2)$ dalam banyak kasus praktis. Kompleksitas waktu rata-rata tetap $O(n \log n)$.

3. Kelayakan pada Singly Linked List

Implementasi Median-of-Three pada singly linked list kurang layak karena keterbatasan berikut:

- a) Akses elemen tengah (mid) membutuhkan traversal $O(n/2)$ karena tidak ada indeks acak pada linked list.
- b) Akses elemen terakhir (last) juga membutuhkan traversal $O(n)$ karena harus mengikuti pointer `.next` dari awal.
- c) Secara total, hanya untuk menentukan tiga kandidat pivot, biayanya adalah $O(n)$. Ditambah biaya partisi $O(n)$, overhead menjadi 2x lebih besar per level rekursi.

Alternatif yang lebih praktis untuk linked list adalah memilih elemen pertama atau tengah saja (ditemukan dengan fast-slow pointer) sebagai pivot, atau menggunakan Merge Sort yang memang lebih alami untuk linked list karena tidak memerlukan akses acak.

Soal d. Batas Teoretis & Paradigma Sorting

1. Mengapa Radix Sort Tidak Kontradiktif dengan $\Omega(n \log n)$

Batas bawah $\Omega(n \log n)$ untuk comparison sort dibuktikan melalui decision tree argument: setiap algoritma sorting berbasis perbandingan dapat direpresentasikan sebagai pohon keputusan biner. Jumlah daun (leaf) minimal adalah $n!$ (semua kemungkinan permutasi input). Karena pohon biner dengan tinggi h memiliki paling banyak 2^h daun, maka $h \geq \log_2(n!) = \Omega(n \log n)$.

Radix Sort tidak melanggar batas ini karena Radix Sort bukan comparison sort. Radix Sort tidak pernah membandingkan dua kunci secara langsung untuk menentukan urutan relatifnya. Radix Sort mengeksploitasi representasi internal kunci (digit/bit) untuk mendistribusikan elemen ke bucket. Model computation-nya berbeda dari model comparison yang menjadi dasar bukti $\Omega(n \log n)$, sehingga batas bawah tersebut tidak berlaku untuk Radix Sort.

2. Dua Asumsi Implisit yang Membuat Radix Sort Melampaui Comparison Sort

Asumsi pertama: Kunci memiliki jumlah digit d yang kecil dan konstan. Radix Sort bekerja dalam d pass, dan jika d adalah konstanta (tidak bergantung pada n), maka kompleksitas waktu adalah $O(d \times n) = O(n)$ yang bersifat linear. Namun jika $d = O(\log n)$ (misalnya kunci berupa bilangan 32-bit untuk $n = 2^{32}$ elemen), maka $O(d \times n) = O(n \log n)$ dan keunggulan Radix Sort menghilang.

Asumsi kedua: Jumlah kemungkinan nilai per digit k (radix) adalah kecil dan konstan. Umumnya $k = 10$ (desimal) atau $k = 256$ (byte). Jika k terlalu besar, ukuran bucket array

menjadi besar dan biaya inisialisasi serta akses memori meningkat signifikan. Dengan k konstan, biaya per pass tetap $O(n + k) = O(n)$.

Kesimpulannya: Radix Sort hanya melampaui batas comparison sort ketika domain kunci terbatas dan terstruktur (d dan k kecil). Pada data umum dengan kunci acak berukuran besar, Radix Sort tidak selalu lebih cepat dalam praktik meskipun secara asimtotik terlihat lebih baik.

BAGIAN II: TEORI & ANALISIS ALGORITMA BINARY TREE

Soal a. Pohon Ekspresi & Traversal

1. Langkah-langkah `_buildTree()` untuk Ekspresi $((8 * 5) + (9 / (7 - 4)))$

Fungsi `_buildTree()` menerima antrian token yang dihasilkan dari tokenisasi ekspresi karakter per karakter. Proses pembangunan pohon berlangsung secara rekursif berdasarkan struktur tanda kurung sebagai berikut:

Token input (setelah tokenisasi): [(, (, 8, *, 5,), +, (, 9, /, (, 7, -, 4,),),)]

Langkah rekursi:

1. Konsumsi '(' pertama -> panggil rekursi untuk subpohon kiri.
2. Di rekursi kiri: konsumsi '(' -> rekursi lebih dalam.
 - a. Konsumsi '8' -> node leaf dengan nilai 8, return.
 - b. Konsumsi '*' -> jadikan operator, simpan sebagai root subpohon.
 - c. Konsumsi '5' -> node leaf nilai 5, jadikan anak kanan.
 - d. Konsumsi ')' -> selesai, return node (*) dengan left=8, right=5.
3. Konsumsi '+' -> jadikan root pohon utama.
4. Rekursi untuk subpohon kanan:
 - a. Konsumsi '(' -> rekursi.
 - b. Konsumsi '9' -> leaf nilai 9.
 - c. Konsumsi '/' -> operator.
 - d. Rekursi untuk anak kanan '/':
 - i. Konsumsi '(' -> rekursi.
 - ii. Konsumsi '7' -> leaf nilai 7.
 - iii. Konsumsi '-' -> operator.
 - iv. Konsumsi '4' -> leaf nilai 4.
 - v. Konsumsi ')' -> return node (-) dengan left=7, right=4.
 - e. Konsumsi ')' -> return node (/) dengan left=9, right=(-).
5. Konsumsi ')' terakhir -> selesai, return root (+) dengan left=(*), right=(/).

Struktur pohon akhir: root=(+), left=(* dengan leaf 8 dan 5), right=(/ dengan leaf 9 dan (- dengan leaf 7 dan 4)).

2. Mengapa Postorder Menghasilkan Postfix Secara Otomatis

Traversal postorder mengunjungi node dalam urutan: kiri -> kanan -> root. Urutan kunjungan ini secara alami menghasilkan notasi postfix karena operator selalu ditulis setelah kedua operandnya selesai dikunjungi, persis sesuai definisi notasi postfix (Reverse Polish Notation).

Postorder pada pohon di atas menghasilkan: $8\ 5\ * \ 9\ 7\ 4\ - \ / \ +$, yang merupakan ekspresi postfix yang valid dan dapat dievaluasi langsung menggunakan stack tanpa perlu tanda kurung.

Sebaliknya, inorder mengunjungi node dalam urutan: kiri -> root -> kanan, yang menghasilkan notasi infix. Pada infix, tanda kurung diperlukan secara eksplisit untuk menentukan prioritas operasi yang ambigu, seperti pada Listing 13.7 yang menyisipkan karakter '(' sebelum subpohon dan ')' sesudahnya saat traversal.

3. Kedalaman Stack Rekursi `_buildString()` untuk Pohon Tinggi h

Fungsi `_buildString()` bersifat rekursif dan dipanggil untuk setiap node dalam pohon. Kedalaman stack rekursi maksimum bergantung pada jalur terpanjang dari root ke leaf, yaitu tinggi pohon h .

Pada setiap pemanggilan rekursif, satu frame ditambahkan ke stack. Dalam kasus terburuk (pohon miring/skewed), seluruh pohon membentuk satu jalur linear sehingga $h = n-1$. Dalam kasus terbaik (pohon seimbang sempurna), $h = \log_2(n)$.

Kesimpulan: Kedalaman maksimum stack rekursi untuk `_buildString()` adalah $O(h)$. Jika pohon seimbang, maka $O(\log n)$; jika pohon miring, maka $O(n)$.

Soal b. Struktur Heap & Pemetaan Array

1. Mengapa Rumus Indeks Hanya Valid pada Complete Binary Tree

Rumus $\text{parent} = (i-1) // 2$, $\text{left} = 2*i+1$, dan $\text{right} = 2*i+2$ hanya valid pada complete binary tree karena representasi array dari sebuah pohon mengandalkan pemetaan posisi yang konsisten dan tanpa celah (gap).

Pada complete binary tree, setiap level terisi penuh dari kiri ke kanan tanpa ada node yang terlewat. Hal ini menjamin bahwa ketika node dengan indeks i disimpan di `arr[i]`, maka anak kirinya pasti berada di `arr[2*i+1]` dan anak kanannya di `arr[2*i+2]` secara berurutan dan kontinyu.

Jika pohon tidak complete (ada node yang hilang di tengah), maka pemetaan indeks akan menghasilkan posisi yang salah karena slot kosong (null node) tidak direpresentasikan dalam array, menyebabkan pergeseran indeks seluruh node di bawahnya.

2. Proses `sift-down()` Setelah Ekstraksi Akar

Setelah akar (elemen maksimum) diekstrak dari max-heap:

1. Elemen terakhir dari heap (`arr[n-1]`) dipindahkan ke posisi akar (`arr[0]`).
2. Ukuran heap dikurangi 1 (`heap_size = n-1`).
3. `sift-down()` dipanggil dari indeks 0 untuk memulihkan heap order property:

- a. Bandingkan $\text{arr}[\text{idx}]$ dengan anak kiri ($\text{arr}[2*\text{idx}+1]$) dan anak kanan ($\text{arr}[2*\text{idx}+2]$).
- b. Tentukan indeks largest: indeks yang memiliki nilai terbesar di antara ketiganya.
- c. Jika $\text{largest} \neq \text{idx}$, tukar $\text{arr}[\text{idx}]$ dengan $\text{arr}[\text{largest}]$, lalu lanjutkan sift-down dari posisi largest.
- d. Ulangi hingga $\text{arr}[\text{idx}] \geq$ kedua anaknya atau idx sudah mencapai daun.

3. Jumlah Perbandingan Maksimum dalam Satu sift-down()

Pada setiap level dalam sift-down, dilakukan tepat 2 perbandingan: satu untuk menentukan anak terbesar antara kiri dan kanan, dan satu untuk membandingkan anak terbesar dengan node saat ini. Tinggi heap berbasis array berukuran n adalah $\text{floor}(\log_2(n))$.

Oleh karena itu, jumlah perbandingan maksimum dalam satu pemanggilan sift-down() adalah $2 \times \text{floor}(\log_2(n)) = O(\log n)$.

Soal c. Heapsort In-Place vs Simple Heapsort

1. Kompleksitas Ruang Tambahan

Simple Heapsort (Listing 13.11) mengekstrak elemen satu per satu dari max-heap ke array terpisah yang baru. Hal ini membutuhkan ruang tambahan $O(n)$ untuk array output. Dengan demikian, total ruang adalah $O(n)$ di luar input.

In-Place Heapsort (Listing 13.12) tidak menggunakan array tambahan. Proses sorting dilakukan langsung pada array input dengan menukar elemen $\text{arr}[0]$ (akar heap) ke $\text{arr}[\text{end}]$, lalu mengurangi heap_size dan memanggil sift-down. Hanya dibutuhkan beberapa variabel indeks ($O(1)$ extra space).

2. Pola Akses Memori (Cache Locality)

Simple Heapsort memiliki akses memori yang lebih prediktif saat menulis ke array output (sequential write), tetapi akses ke heap bersifat acak (random access) karena sift-down melompat dari root ke child secara tidak berurutan.

In-Place Heapsort memiliki pola akses yang sama acaknya karena sift-down tetap melakukan akses non-sequential ke dalam array yang sama. Akibatnya, cache miss rate keduanya serupa dan relatif tinggi, menjadikan heapsort secara umum memiliki cache locality yang lebih buruk dibandingkan Merge Sort atau Quick Sort pada data acak.

3. Risiko Overflow pada Sistem RAM Terbatas

Simple Heapsort berisiko lebih tinggi mengalami out-of-memory pada sistem embedded dengan RAM sangat terbatas karena membutuhkan array output berukuran n sebagai ruang tambahan. Pada Modul A dengan 10^6 elemen integer (misalnya 4 byte per elemen), dibutuhkan sekitar 4 MB tambahan.

In-Place Heapsort hampir tidak berisiko overflow karena hanya menggunakan beberapa variabel tambahan $O(1)$, menjadikannya pilihan yang jauh lebih aman untuk sistem embedded.

4. Mengapa In-Place Heapsort Tetap $O(n \log n)$

Meskipun in-place heapsort melakukan swap berulang pada array yang sama, kompleksitas waktunya tetap $O(n \log n)$ karena:

- a) Fase build-heap: membangun max-heap dari $n/2$ node daun ke atas. Setiap pemanggilan sift-down memakan $O(\log n)$. Total: $O(n)$ (dapat dibuktikan secara matematis bahwa total work pada build-heap adalah $O(n)$, bukan $O(n \log n)$).
- b) Fase extraction: dilakukan $n-1$ kali swap diikuti sift-down $O(\log n)$ per iterasi. Total: $O(n \log n)$.

Operasi swap sendiri adalah $O(1)$ dan tidak mengubah asimtotik. Dengan demikian, total kompleksitas waktu in-place heapsort adalah $O(n) + O(n \log n) = O(n \log n)$.

Soal d. Batas Teoretis & Decision Tree

1. Mengapa Heapsort Tidak Melanggar Batas $\Omega(n \log n)$

Heapsort adalah comparison sort karena seluruh pengambilan keputusan urutan elemen didasarkan pada operasi perbandingan ($>$, $<$, $>=$). Oleh karena itu, batas bawah $\Omega(n \log n)$ berlaku untuk heapsort.

Heapsort tidak melanggar batas ini; justru heapsort mencapainya secara asimtotik dengan kompleksitas waktu $O(n \log n)$ dalam semua kasus (best, average, worst). Struktur pohon yang digunakan (heap) bukan alasan heapsort bisa lebih cepat dari batas tersebut, melainkan hanya merupakan cara efisien untuk mengorganisasi perbandingan agar setiap elemen diproses dalam $O(\log n)$ waktu.

2. Properti Null Link pada Decision Tree untuk Deteksi Token Tidak Valid

Pada pohon keputusan seperti decision tree untuk decoding Morse Code, setiap node merepresentasikan satu simbol yang mungkin dan memiliki dua anak: left (titik/dot) dan right (garis/dash). Jalur dari root ke sebuah node merepresentasikan urutan sinyal Morse untuk simbol tersebut.

Properti null link digunakan sebagai berikut: jika pada proses decoding, pointer mencapai node None (null), berarti tidak ada simbol yang valid untuk urutan sinyal yang diberikan. Kondisi ini langsung menandai token sebagai tidak valid tanpa perlu pemeriksaan tambahan.

Dengan demikian, null link berfungsi sebagai implicit error state: tidak perlu menyimpan daftar urutan yang tidak valid secara eksplisit karena ketiadaan node sudah merepresentasikan ketidakvalidan.

3. Mengapa Rekursif Lebih Cocok dari Iteratif untuk Pohon Keputusan

Pendekatan rekursif lebih cocok untuk membangun pohon keputusan karena:

- a) Struktur masalah bersifat self-similar: setiap subpohon memiliki struktur yang identik dengan pohon keseluruhan, hanya berbeda tingkat kedalaman dan simbol yang dikodekan.
- b) Manajemen state otomatis: call stack secara otomatis menyimpan posisi saat ini dalam pohon, sedangkan pendekatan iteratif harus secara eksplisit mengelola stack manual yang lebih kompleks.
- c) Kode lebih ekspresif: logika rekursif mencerminkan langsung definisi pohon (node = data + left subtree + right subtree), membuat kode lebih mudah dibaca, diverifikasi, dan di-debug.
- d) Depth-first traversal alami: pembangunan pohon keputusan memerlukan eksplorasi mendalam (depth-first) sebelum berpindah ke cabang lain, yang merupakan pola alami rekursi.

Pendekatan iteratif meskipun memungkinkan, membutuhkan pengelolaan eksplisit stack dan state yang membuat kode lebih panjang dan rentan kesalahan, terutama pada struktur pohon yang dalam dan tidak seimbang.